E3 · Especificação do Código Java — Kruskal

Grupo 1 · Tema 4: Árvore geradora mínima

Objetivo

Um projeto Java que executa sozinho o mesmo Kruskal que o grupo faz à mão: recebe pontos e ligações com peso, escolhe as ligações que conectam todos os pontos com a menor soma possível e imprime o resultado.

Não usa tela gráfica, banco de dados nem Google Maps. As distâncias dos ecopontos foram medidas manualmente e entram direto no código.

Regra de ouro do guia: a saída do programa precisa ser igual à da simulação à mão. Se o código mostrar um resultado e o slide outro, o grupo perde pontos.

Fluxo do programa

A ordem é fixa: primeiro os dois grafos obrigatórios, intactos; só depois o modo interativo.

1. **Grafo A–F.** Monta as 8 arestas do professor (tratadas como não dirigidas, peso em horas), roda o Kruskal e imprime o resultado. Total esperado: 21 h.
2. **Grafo dos ecopontos.** Monta as 10 ligações com as médias em km, roda o Kruskal e imprime. Total esperado: 53,9 km.
3. **Modo interativo (opcional).** Oferece ao usuário adicionar novos pontos e ligações e rodar o Kruskal de novo, sobre uma cópia do grafo dos ecopontos.

Os passos 1 e 2 são o que vale nota. O passo 3 é um extra para mostrar o cadastro funcionando.

Classes

Classe	Responsabilidade
`Ponto`	Representa um local do grafo: nome, endereço e índice. Cada ecoponto (e cada vértice A–F) é um objeto `Ponto`, criado com `new`, inclusive a partir do que o usuário digitar. No A–F o endereço fica vazio.

Classe	Responsabilidade
`Aresta`	Liga dois objetos `Ponto` (origem e destino) com um peso. Equivale a uma linha da tabela de distâncias. Implementa `Comparable` para ordenar pelo peso.
`Grafo`	Guarda os pontos num `Map<String, Ponto>` (chave = nome normalizado) e a lista de arestas. Métodos `obterOuCriarPonto(nome, endereco)` e `adicionarAresta(origem, destino, peso)`.
`UniaoBusca`	Os "grupinhos". `encontrar(x)` diz em qual grupo está um ponto (pelo seu índice); `unir(a, b)` junta dois grupos. É o que detecta ciclo.
`Kruskal`	Ordena as arestas, testa uma a uma com a `UniaoBusca`, aceita ou descarta e para ao chegar em n − 1 arestas. Devolve as aceitas, as descartadas e o custo total.
`Main`	Monta os dois grafos, chama o Kruskal, imprime os resultados e oferece o modo interativo.

Como o Kruskal decide cada aresta: se `encontrar(origem) != encontrar(destino)`, as pontas estão em grupos diferentes, então aceita e chama `unir`. Se forem iguais, já existe caminho entre elas, então descarta porque formaria ciclo.

Desempate: quando duas arestas têm o mesmo peso (A–B e D–E, ambas 4 h), a ordem é alfabética, igual à da simulação à mão.

Complexidade: O(m log m), dominada pela ordenação das m arestas. A `UniaoBusca` é praticamente constante por operação.

Como o código detecta o ciclo (união-busca)

O Kruskal é o mesmo da mão: ordenar as ligações da menor para a maior e ir adicionando, descartando as que fecham loop. A união-busca só responde, dentro do código, à pergunta "essa ligação fecha loop?". À mão, a gente olha o desenho; o computador não enxerga desenho, então usa grupos.

A ideia: cada ponto tem um número de grupo. No começo, cada um está sozinho:

```
ponto:  A  B  C  D  E  F
grupo:  0  1  2  3  4  5
```

Para testar uma ligação, compara-se o grupo das duas pontas:

- **grupos diferentes:** não fecha loop, então aceita e junta os dois grupos num só;
- **mesmo grupo:** já existe caminho entre eles, então fecha loop e descarta.

Ligação	Grupos das pontas	Decisão	Situação depois
A–C	0 e 2	Aceita, junta	A, C no grupo 0
C–E	0 e 4	Aceita, junta	A, C, E no grupo 0
A–B	0 e 1	Aceita, junta	A, B, C, E no grupo 0
D–E	3 e 0	Aceita, junta	A, B, C, D, E no grupo 0
B–C	0 e 0	Mesmo grupo: loop, descarta	sem mudança
E–F	0 e 5	Aceita, junta	todos no grupo 0

Para que serve o índice do `Ponto`: o vetor de grupos usa posições numéricas (grupo[0], grupo[1]...), mas os pontos têm nomes. O índice é a posição de cada ponto nesse vetor: Guanabara → índice 0 → grupo[0], Faiçalville → índice 1 → grupo[1], e assim por diante.

Versão mínima em Java:

```
int[] grupo = new int[n];
for (int i = 0; i < n; i++) grupo[i] = i;   // cada um no seu grupo

for (Aresta a : arestasOrdenadas) {
    int g1 = grupo[a.getOrigem().getIndice()];
    int g2 = grupo[a.getDestino().getIndice()];

    if (g1 == g2) {
        // fecha loop -> descarta
    } else {
        // aceita e junta: todo mundo do grupo g2 passa para o g1
        for (int i = 0; i < n; i++) {
            if (grupo[i] == g2) grupo[i] = g1;
        }
    }
}
```

Essa versão simples é suficiente para 5 ou 6 pontos. A classe `UniaoBusca` com `encontrar` e `unir` faz a mesma coisa de forma mais eficiente. Na arguição, o importante é saber explicar a ideia dos grupos.

Cadastro de pontos

Cada local é um objeto da classe `Ponto`. Não existe uma classe por endereço: é uma classe só, com uma instância para cada local.

```
public class Ponto {
    private final String nome;      // "São José"
    private final String endereco;  // "Rua Frei Nazareno Confaloni..."
    private final int indice;       // posição usada pela UniaoBusca

    public Ponto(String nome, String endereco, int indice) {
        this.nome = nome;
        this.endereco = endereco;
        this.indice = indice;
    }
    // getters
}
```

O `Grafo` guarda os pontos num mapa cuja chave é o **nome normalizado**: sem espaços nas pontas, em maiúsculas e com `_` no lugar dos espaços. Assim "são josé", "São José" e "SÃO JOSÉ " viram a mesma chave e não duplicam o ponto.

```
private Map<String, Ponto> pontos = new HashMap<>();

private String normalizar(String nome) {
    return nome.trim().toUpperCase().replace(" ", "_");
}

public Ponto obterOuCriarPonto(String nome, String endereco) {
    String chave = normalizar(nome);
    // já existe: devolve o mesmo objeto; não existe: cria um novo com o
próximo índice
    return pontos.computeIfAbsent(chave,
        k -> new Ponto(nome, endereco, pontos.size()));
}
```

O índice dentro de cada `Ponto` é o que a `UniaoBusca` usa por baixo dos panos. É assim que o nome real vira número na estrutura.

Opcional: para que "Sao Jose" e "São José" também sejam o mesmo ponto, remover os acentos com `java.text.Normalizer` antes de passar para maiúsculo.

Por que não enum: valores de enum são fixos na compilação e não podem ser criados a partir do que o usuário digita. Com a classe `Ponto`, os 5 ecopontos fixos e os pontos novos do modo interativo são o mesmo tipo de objeto.

Os dois grafos obrigatórios são cadastrados assim, direto no `Main`, sem digitação. Isso evita erro de digitação na apresentação ao vivo e garante que a saída bata com a simulação.

Modo interativo (opcional)

Depois das duas execuções fixas, o programa oferece um menu para o usuário:

1. Adicionar um novo ponto, informando o nome.
2. Ligar esse ponto a um ou mais pontos existentes, informando a distância de cada ligação.
3. Rodar o Kruskal de novo e ver a nova rede.

Não é preciso informar a distância para todos os pontos. O Kruskal funciona em qualquer grafo conectado, não só em grafos em que tudo liga com tudo.

Regra: todo ponto novo precisa ter pelo menos uma ligação. Duas proteções:

- **Na entrada:** o programa não aceita cadastrar um ponto sem nenhuma ligação.
- **Na saída:** se o Kruskal terminar com menos de n − 1 arestas aceitas, avisa que o grafo não está conectado e que não existe árvore geradora.

O modo interativo trabalha sobre uma **cópia** do grafo dos ecopontos, para que o resultado original de 53,9 km não seja alterado.

Saída esperada

O programa imprime as aceitas e também as descartadas, na ordem em que são testadas. Assim dá para comparar linha por linha com os slides da simulação.

```
=== Grafo A-F ===
Aceita:     A - C (2 h)
Aceita:     C - E (3 h)
Aceita:     A - B (4 h)
Aceita:     D - E (4 h)
Descartada: B - C (5 h) -> formaria ciclo
Aceita:     E - F (8 h)
Custo total: 21 h

=== Ecopontos Comurg ===
Aceita:     São José - Parque Eldorado Oeste (2,7 km)
Aceita:     Faiçalville - Campos Dourados (11,4 km)
Aceita:     Faiçalville - São José (13,95 km)
Descartada: Faiçalville - Parque Eldorado Oeste (16,1 km) -> formaria ciclo
Descartada: São José - Campos Dourados (18,9 km) -> formaria ciclo
Descartada: Campos Dourados - Parque Eldorado Oeste (21,05 km) -> formaria
ciclo
Aceita:     Guanabara - Faiçalville (25,85 km)
Custo total: 53,9 km
```

Nos dois casos o programa para ao atingir n − 1 arestas aceitas (5 no A–F, 4 nos ecopontos). As arestas restantes nem chegam a ser testadas.

Cuidados

- **Nomes iguais em todo lugar.** "Faiçalville" e "Faicalville" viram dois vértices diferentes. Usem sempre a mesma grafia, com acento.
- **Ponto decimal no código.** Em Java o número é `25.85`. Na impressão, formatem com vírgula (por exemplo, `String.format(new Locale("pt", "BR"), ...)`) para bater com os slides.
- **Grafo não dirigido.** Cada ligação vale nos dois sentidos. No Kruskal basta cadastrar a aresta uma vez.
- **Nomes em português** e comentários nos trechos importantes, conforme pede o guia.

Checklist antes da entrega (18/11)

- [ ] Compila e roda sem erros.
- [ ] Saída do A–F igual à simulação à mão: 5 arestas, total de 21 h.
- [ ] Saída dos ecopontos igual à simulação: 4 ligações, total de 53,9 km.
- [ ] Empate de 4 h resolvido em ordem alfabética (A–B antes de D–E).
- [ ] Modo interativo não altera o resultado original.

- [ ] Aviso de grafo desconectado funcionando.
- [ ] Projeto em .zip ou link do repositório, com o nome dos 7 integrantes.